Leibniz FH

Wirtschaftsinformatik (B. Sc.)
Modul 20.1 (Vertiefung Entwicklung)
Sommersemester 2026

Anwendung erweiterter Softwareentwicklung

1. Einleitung

Dr. Peter Lohmann

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Leibniz FH

1.1. Organisatorisches

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Vorstellung

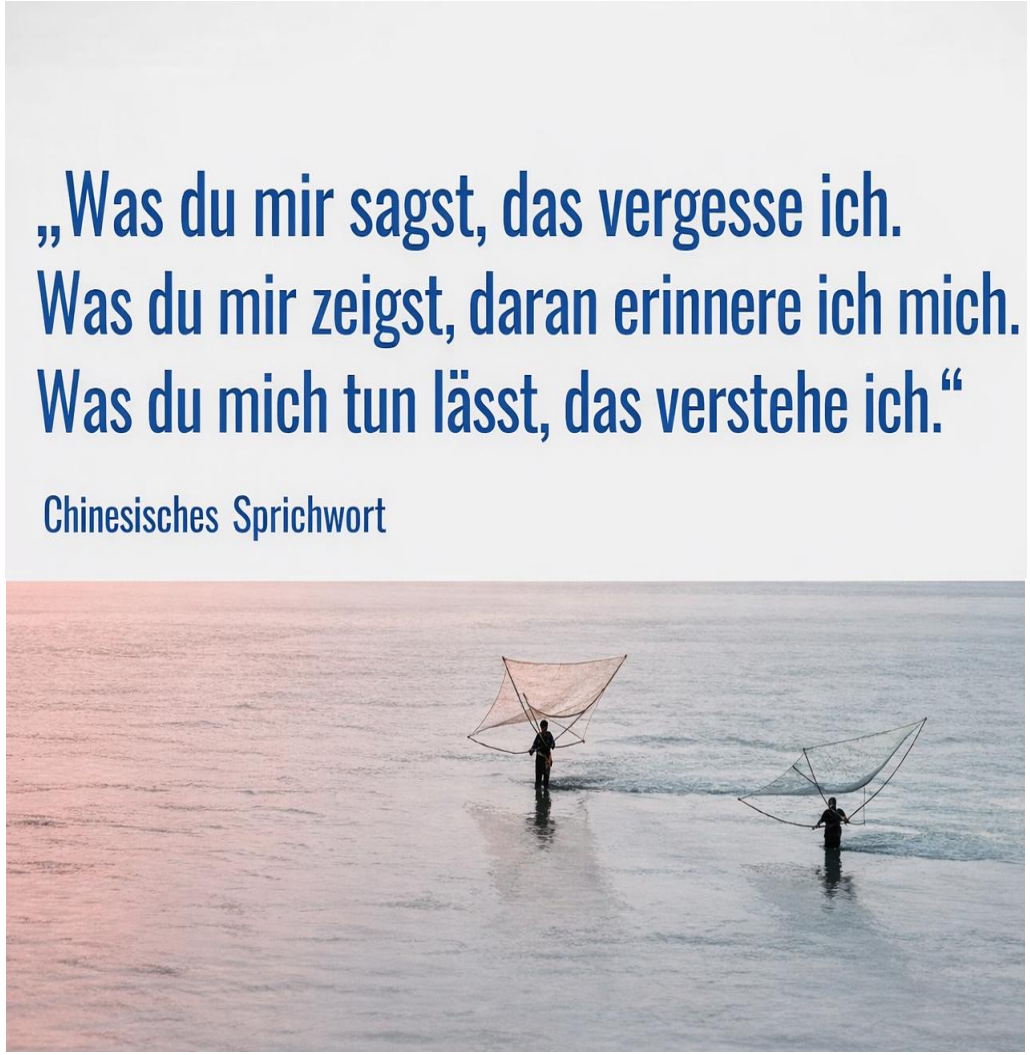
- ✓ Peter Lohmann
  - Diplom in Mathematik und Informatik
  - Promotion in theoretischer Informatik - Logik und Komplexität
  - Software-Entwickler und -Architekt, Data-Engineer, vor allem C++ und Python
- ✓ Kommunikationswege
  - Peter.Lohmann.doz@leibniz-fh.de oder direkt peter@lohmann-web.net
  - Stud.IP

Aufbau der Lehrveranstaltung

- ✓ Interaktive Vorlesung
  - Inhalte werden gemeinsam erarbeitet
  - Fragen und Diskussionen sind ausdrücklich erwünscht
- ✓ Gruppenarbeit an Programmieraufgaben
  - Anwendung der Inhalte direkt in kleinen Teams
  - Fokus auf gemeinsames Lösen und Besprechen von Problemen
- ✓ Regelmäßige Nachbereitung der Vorlesung
  - Inhalte zeitnah wiederholen und an Beispielen nachvollziehen
  - offene Punkte möglichst früh klären - zu Beginn der nächsten Vorlesung
  - keine automatische Wiederholung in der Vorlesung

Programmieren muss man üben

- ✓ Lernen durch eigenes Anwenden
  - Konzepte werden klar durch eigenes Programmieren
  - Fehler und Rückfragen gehören dazu
- ✓ Nachhaltiges Lernen braucht Praxis
  - Regelmäßiges Üben ist wichtiger als Wiedererkennen
  - Ziel: handlungsfähiges statt passives Wissen
- ✓ Heute: Technische Grundlage schaffen
  - Entwicklungsumgebung und Tools einrichten



Leibniz FH

1.2. Lernziele

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Programmierkonzepte

- ✓ Konzepte, die in der Praxis wirklich gebraucht werden
  - Ziel ist ein tragfähiges Verständnis statt isolierter Einzelfakten
- ✓ Fokus auf objektorientierte Softwareentwicklung
  - zentrale Ideen der Objektorientierung werden schrittweise aufgebaut
  - auch Design und Architektur-Prinzipien

As long as there were no machines, programming was no problem at all;

when we had a few weak computers, programming became a mild problem,

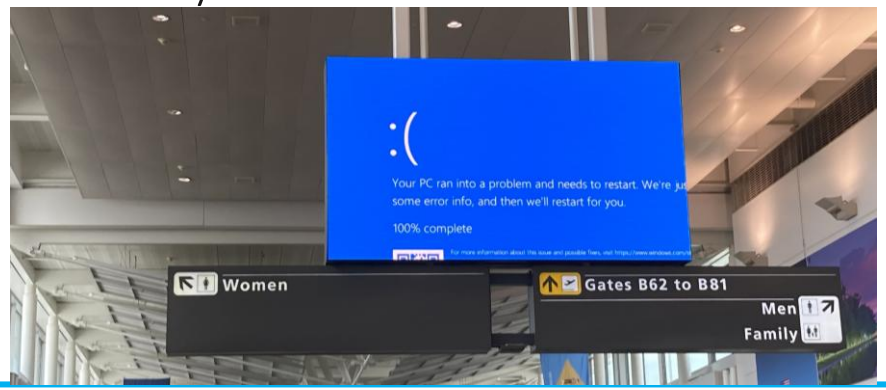
and now we have gigantic computers, programming has become a gigantic problem.

Edsger W. Dijkstra (1930-2002)
ACM Turing Award Lecture 1972

Wenn es schief geht

- ✓ Ariane 5 – Rakete (1996)
  - ungeprüfte Wiederverwendung von Ariane-4-Software
  - falsche Wertebereichsannahmen → Overflow
    → Risiko: Software-Reuse ohne Kontextprüfung

- ✓ Therac-25 – Bestrahlungsgerät (1980er)
  - Race Conditions + fehlende Hardware-Sicherungen
  - Überdosen mit schweren Folgen für Patienten
    → Risiko: fehlende Absicherung sicherheitskritischer Systeme



- ✓ CrowdStrike – Sicherheitsupdate (2024)
  - fehlerhafte globale Konfiguration
  - kein gestuftes Rollout
    → Risiko: zentrale Updates ohne Fallback-Strategie

Produktiv arbeiten

- ✓ Ein produktiver Programmier-Workflow ist Teil der Lernziele
  - gute *Tools* (IDE, Git, Paketverwaltung, …) und klare *Prozesse* (Testen, Reviews, …) sparen Zeit und vermeiden Fehler
  - Programmieren besteht nicht nur aus Schreiben von Code!
- ✓ Bedeutung einer effizienten Entwicklungsumgebung
  - *Editor*, *Terminal* und *Debugger* sollen sicher beherrscht werden
- ✓ Hilfreiche Tools und Abläufe kennenlernen
  - wir üben sinnvolle Routinen für Entwicklung und Fehlersuche ein
  - Ziel ist ein state-of-the-art Arbeitsstil

Überfachliche Kompetenzen

- ✓ *Teamarbeit* gehört zum Lernprozess dazu
  - viele Aufgaben lassen sich besser gemeinsam analysieren und lösen
  - Zusammenarbeit ist auch in der späteren Praxis zentral
- ✓ *Fachliche Kommunikation* wird bewusst eingeübt
  - Lösungen müssen verständlich erklärt und begründet werden können
  - dazu gehört auch der präzise Umgang mit Begriffen und Notationen

Leibniz FH

1.3. Grobstruktur

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Inhalte

- ✓ Python-Grundlagen
- ✓ Laufzeitmodell und Internals
- ✓ Entwicklungsumgebung
- ✓ Software-Design
- ✓ Parallelisierung und Persistenz

Python-Grundlagen

- ✓ Imperative Grundlagen
  - Typen und Ausdrücke
  - Schleifen
  - Funktionen
- ✓ Objektorientierte Sprachkonzepte
  - Vererbung und Polymorphie
  - Interfaces
  - Referenzen vs Kopien
- ✓ Reflektion
  - Meta-Informationen über Programm-Konstrukte

Laufzeitmodell und Internals

- ✓ Imports
  - Laden von (externen) Bibliotheken
- ✓ Exceptions
  - Fehlerbehandlung als Teil des Programmdesigns
- ✓ Interpreter und Compiler
  - Unterschiede beim Übersetzen und Ausführen von Programmen
- ✓ Speicherverwaltung
  - Garbage Collection und Vergleich zu C++

Entwicklungsumgebung

- ✓ IDE und Werkzeuge
  - Effizient arbeiten, navigieren und Code ausführen
- ✓ Versionskontrolle und Bibliothek-Abhängigkeiten
  - Arbeitsstände sichern und Projekte reproduzierbar verwalten
  - Kollaborativ programmieren
- ✓ Debugging
  - Fehler systematisch finden und verstehen

Software-Design

- ✓ SOLID und DRY
  - Prinzipien für wartbaren und gut strukturierten Code
- ✓ Abstraktionsebenen
  - passende Verantwortlichkeiten und saubere Schnittstellen
- ✓ Entwurfsfragen im Code
  - Begründbarkeit und Nachvollziehbarkeit

Parallelisierung und Persistenz

- ✓ Concurrency Models
  - Prozesse, Threads und Coroutines
- ✓ Synchronisation
  - Zusammenarbeit gleichzeitig laufender Teile koordinieren
- ✓ Serialisierung und ORM
  - Daten zwischen Speicher, Datei und Datenbank übertragen